Bangladesh Army University of Engineering & Technology

Qadirabad, Natore-6431.

Faculty of Electrical and Computer Engineering

Department of Computer Science and Engineering

Course Code: CSE-2104

Course Title: Object Oriented Programming Laboratory

Sessional Plan

Program:	Computer Science and Engineering (CSE)
Course Code & Title:	CSE-2104 Object Oriented Programming Laboratory
Semester:	Summer-2025
Level:	2nd Year 1st Semester
Credit Hour:	1.50 Hr.
	Md. Atikur Rahman Lecturer, Dept. of CSE
	Mehedi Hasan Lecturer, Dept. of CSE
Class Hours:	Monday: 11:40 AM-02.55 PM Tuesday: 11:40 AM-02.55 PM
Consultation Hours:	Tuesday: 09:15 AM to 10:15 AM
E-mail:	atik.cse.pust35@gmail.com (Md. Atikur Rahman) mehedishagor203303@gmail.com (Mehedi Hasan)
Mobile:	+880 1774320831(Md. Atikur Rahman) +880 1773367762 (Mehedi Hasan)
Rationale:	Object-oriented programming is a programming paradigm that includes or relies on the concept of objects, encapsulated data structures that have properties and functions and which interact with other objects
Pre-requisite (if any):	N/A
Objectives:	
1. To achieve a basic idea on Object Oriented Programming	
2. Present object-oriented aspects of C++	
3. To learn programming with C++	

Course Content:

Basic OOP: Classes and objects, encapsulation, access specifiers, static and non-static members; Constructors,Destructors and Copy Constructors; Array of objects, object pointers, and object references; In-line functions, friend functions, static functions.

Inheritance: Single, multiple and multilevel inheritances

Polymorphism: overloading, abstract classes, virtual functions and overriding; Exception Handling; Object Oriented I/O, Template functions and classes, Concept of Namespaces, Overview of Standard Template Library(Vectors & Iterators), Multi-threaded Programming, Abstract Data Types

Text Book(s):

1. C++: A Beginner's Guide, Herbert Schildt, McGraw-Hill/Osborne, 3rd Edition, ISBN: 9780071634663
2. C++: The Complete Reference, Herbert Schildt, McGraw Hill Professional, 4th Edition, ISBN: 9780072226805
3. Object Oriented Programming with C++, Nabajyoti Barkakati, Sams, 1st Edition, ISBN: 9780672228001

Reference Book(s):

1. Object Oriented Programming with C++, Balagurusamy, Tata McGraw-Hill Education, 1st Edition, ISBN: 9780070669079
1. Object-Oriented Programming in C++, Robert Lafore, Pearson Education, 4th Edition, ISBN: 9780132714297

*** Additional reading material: PowerPoint Slides & PDFs provided by the teachers**

Mapping CO-PO, Bloom's Level, Knowledge Profile, Complex Engineering Problem (WP) and Complex Engineering Activities (EA)

COs	CO Statements	PO	KP	WP	EA	Bloom's Level	Delivery methods and activities	Assessment Tools
CO1	Apply object-oriented programming principles to design and implement small problem solving using multiple interacting objects.	1	3	1	1	C2, C3, C4	Lectures, Textbook, PPT, PDF White Board Writing	Class Participation/ Observation Homework/ Assignment, Class Test, Semester Final Examination (Written)
CO2	Demonstrate appropriate programming style and structure to ensure readability, maintainability, and scalability of object-oriented programs.	2	3	1	1	C2, C3, C5		
CO3	Analyze and compare alternative object-oriented constructs, algorithmic designs, and data structures to determine the most efficient solution.	5	6	1	1	C3, C4 C5		
CO4	Develop, test, and document object-oriented applications using professional tools and practices, creating complete software packages.	8	6	1	1	C3, C5, C6		

Legends: CO-Course Outcomes PO-Program Outcomes, KP-Knowledge Profile

Bloom's Level: Cognitive (C1-C6), Psychomotor(P1-P7), Affective(A1-A5)

COs	Program Outcome (PO)											
	1	2	3	4	5	6	7	8	9	10	11	12
CO1	H											
CO2		H										
CO3					M							
CO4								M				

Legends: High(H)=3, Medium(M)=2 and Low(L)=1

Assessment of CO-PO, Bloom's Level, Knowledge Profile, Complex Engineering Problem (WP) and Complex Engineering Activities (EA)

Teaching-learning and Assessment Strategy:

Lab test/Project, Written test/ Quiz, Viva Voce, Attendance, Data Sheet/Home Assignments/ Report, Class Performance/ Continuous Evaluation

Assessment Methods	Description	Grading	CO	Blooms Taxonomy
Online Test -01 (Midterm Lab Assessment)	Online Test -01 (Midterm Lab Assessment)	20%	CO1, CO2	C2, C3, C4, C5
Online Test -02 (Final Lab Assessment)	Online Test -02 (Final Lab Assessment)	30%	CO3, CO4	C3, C4, C5, C6
Viva-Voce/Presentation Skills	Viva Voce	10%	CO2	C2, C3, C5
Report Writing	Report Writing	15%	CO1, CO2	C2, C3
Conduct of Lab test/Lab Performance	Class Performance	25%	CO1, CO2	C2
Total Marks		100%		

Sessional Lecture Plan

Sl. No.	Content of Course	Lecture Hours	Mapped COs	Teaching Aids	Teaching & Assessment Tools
01	Classes and Objects	3 Hrs	CO1, CO2	PowerPoint Slides, Lab Manual, White Board	Sessional Class, Lab Report, Individual/Group Discussion
02	Encapsulation, Access Specifiers, Static and Non-static Members	3 Hrs	CO1, CO2	PowerPoint Slides, Lab Manual, White Board	Sessional Class, Lab Report, Individual/Group Discussion
03	Constructors, Destructors, and Copy Constructors	3 Hrs	CO1, CO2	PowerPoint Slides, Lab Manual, White Board	Sessional Class, Lab Report, Individual/Group Discussion

04	Array of Objects, Object Pointers, Object References, In-line Functions	3 Hrs	CO1, CO2	PowerPoint Slides, Lab Manual, White Board	Sessional Class, Lab Report, Individual/Group Discussion
05	Friend Functions, Static Functions	3 Hrs	CO2, CO3	PowerPoint Slides, Lab Manual, White Board	Sessional Class, Lab Report, Individual/Group Discussion
06	Single Inheritance	3 Hrs	CO2, CO3	PowerPoint Slides, Lab Manual, White Board	Sessional Class, Lab Report, Individual/Group Discussion
07	**Online Test - 01**	1.5 Hrs	CO1, CO2	—	Assessment
08	Multiple Inheritance	3 Hrs	CO2, CO3	PowerPoint Slides, Lab Manual, White Board	Sessional Class, Lab Report, Individual/Group Discussion
09	Function Overloading, Operator Overloading	3 Hrs	CO2, CO3, CO4	PowerPoint Slides, Lab Manual, White Board	Sessional Class, Lab Report, Individual/Group Discussion
10	Abstract Classes, Virtual Functions, Overriding	3 Hrs	CO2, CO3, CO4	PowerPoint Slides, Lab Manual, White Board	Sessional Class, Lab Report, Individual/Group Discussion
11	Exception Handling, Object-Oriented I/O	3 Hrs	CO2, CO3, CO4	PowerPoint Slides, Lab Manual, White Board	Sessional Class, Lab Report, Individual/Group Discussion
12	File Handling, Introduction to STL (Vectors, Iterators)	3 Hrs	CO2, CO3, CO4	PowerPoint Slides, Lab Manual, White Board	Sessional Class, Lab Report, Individual/Group Discussion
13	**Online Test - 02**	1.5 Hrs	CO3, CO4	—	Assessment
14	**Viva Voce**	3 Hrs	CO2, CO3	—	Oral Assessment

Teaching-learning and Assessment Strategy:
Sessional class participations, Lab Reports, Lab Project, Lab Test and Viva-voce.

Minimum Attendance: 60% class attendance is mandatory for a student in order to appear at the finalexamination.

Grading System: As per the approved grading scale of Bangladesh Army University of Engineering & Technology (BAUET), Qadirabad, Natore **(Appendix-II)**

Student's responsibilities:

a. Students must come to the class prepared for the course material covered in the previous class(es).

b. They must submit their lab report and lab project on time.

c. No late or partial report and lab project will be acceptable.There will be no make-up quiz test.

Course Assessment Methods & their Weights:

Assessment Method	(%)
Online Test-1	25
Online Test-2	25
Viva Voce	10
Attendance	10
Continuous Evaluation	10

Class Performance	20
Total Marks	**100**

Program Outcomes and Assessment

Program Outcomes (POs) or graduate attributes are narrower statements that describe what students are expected to know and be able to do by the time of graduation. These statements relate to the knowledge, skills and attitudes acquired by students while progressing through the program. The program must demonstrate that by the time of graduation, students have achieved an acceptable minimum level of certain knowledge, skills and behavioral traits. The BAETE specifically requires that students acquire the following graduate attributes:

Sl.No	PO	Category	Description
1	**PO1**	**Engineering Knowledge**	Apply knowledge of mathematics, natural science, engineering fundamentals and an engineering specialization as specified in K1 to K4 respectively to the solution of complex engineering problems.
2	**PO2**	**Problem Analysis**	Identify, formulate, research literature and analyse complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences and engineering sciences. (K1 to K4).
3	**PO3**	**Design/ Developmentof Solutions**	Design solutions for complex engineering problems and design systems, components or processes that meet specified needs with appropriate consideration for public health and safety, cultural, societal, and environmental considerations. (K5)
4	**PO4**	**Investigation**	Conduct investigations of complex problems using research-based knowledge (K8) and research methods including design of experiments, analysis and interpretation of data, and synthesis of information to provide valid

			conclusions.
5	**PO5**	**Modern Tool Usage**	Create, select and apply appropriate techniques, resources, and modern engineering and IT tools, including prediction and modelling, to complex engineering problems, with an understanding of the limitations. (K6).
6	**PO6**	**The Engineer and Society**	Apply reasoning informed by contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to professional engineering practice and solutions to complex engineering problems. (K7)
7	**PO7**	**Environment and Sustainability**	Understand and evaluate the sustainability and impact of professional engineering work in the solution of complex engineering problems in societal and environmental contexts. (K7)
8	**PO8**	**Ethics**	Apply ethical principles and commit to professional ethics and responsibilities and norms of engineering practice. (K7)
9	**PO9**	**Individual and Team Work**	Function effectively as an individual, and as a member orleader in diverse teams and in multi-disciplinary settings.
10	**PO10**	**Communication**	Communicate effectively on complex engineering activities with the engineering community and with society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
11	**PO11**	**Project Management and Finance**	Demonstrate knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
12	**PO12**	**Life Long Learning**	Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

In addition to incorporating the above-listed POs (graduate attributes), the educational institution may include additional outcomes in its learning programs. An engineering program that aims to attain the abovementioned POs should ensure that its curriculum encompasses all the attributes of the Knowledge Profile (K1 – K8) as presented in Table 1.1 and as included in the PO statements. The ranges of Complex Problem Solving (P1 – P7) and Complex Engineering Activities (A1 – A5) that should be addressed in the program are given in Tables 1.2 and 1.3 respectively.

Table 1.1: Knowledge Profile

Attribute	
K1	A systematic, theory-based understanding of the natural sciences applicable to the discipline
K2	Conceptually based mathematics, numerical analysis, statistics and the formal aspects of computer and information science to support analysis and modeling applicable to the discipline
K3	A systematic, theory-based formulation of engineering fundamentals required in the engineering discipline
K4	Engineering specialist knowledge that provides theoretical frameworks and bodies of knowledge for the accepted practice areas in the engineering discipline; much is at the forefront of the discipline
K5	Knowledge that supports engineering design in a practice area
K6	Knowledge of engineering practice (technology) in the practice areas in the engineering discipline

K7	Comprehension of the role of engineering in society and identified issues in engineering practice in the discipline: ethics and the engineer's professional responsibility to public safety; the impacts of engineering activity; economic, social, cultural, environmental and sustainability
K8	Engagement with selected knowledge in the research literature of the discipline

Table 1.2: Range of Complex Engineering Problem Solving

Attribute	**Complex Engineering Problems** have characteristic P1 and some or all of P2 to P7
Depth of knowledge required	**P1:** Cannot be resolved without in-depth engineering knowledge at thelevel of one or more of K3, K4, K5, K6 or K8 which allows a fundamentals-based, first principles analytical approach
Range of conflicting requirements	**P2:** Involve wide-ranging or conflicting technical, engineering and other issues
Depth of analysis required	**P3:** Have no obvious solution and require abstract thinking, originality in analysis to formulate suitable models
Familiarity of issues	**P4:** Involve infrequently encountered issues
Extent of applicable codes	**P5:** Are outside problems encompassed by standards and codes of practice for professional engineering
Extent of stakeholder involvement and conflicting requirements	**P6:** Involve diverse groups of stakeholders with widely varying needed
Interdependence	**P7:** Are high level problems including many component parts orsub-problems

Table 1.3: Range of Complex Engineering Activities

Attribute	**Complex activities** means (engineering) activities or projects that have some or all of the following characteristics:
Range of resources	**A1:** Involve the use of diverse resources (and for this purpose resources include people, money, equipment, materials, information and technologies)
Level of interaction	**A2:** Require resolution of significant problems arising from interactions between wide-ranging or conflicting technical, engineering or other issues
Innovation	**A3:** Involve creative use of engineering principles and research based knowledge in novel ways
Consequences for society and the environment	**A4:** Have significant consequences in a range of contexts, characterized by difficulty of prediction and mitigation
Familiarity	**A5:** Can extend beyond previous experiences by applying principles-based approaches

Bloom's Taxonomy Domain:

Cognitive Domain		Psychomotor Domain		Affective Domain	
C1	Remembering	**P1**	Perception	**A1**	Receive
C2	Understanding	**P2**	Set	**A2**	Respond
C3	Applying	**P3**	Guided Response	**A3**	Value
C4	Analyzing	**P4**	Mechanism	**A4**	Organize
C5	Evaluating	**P5**	Complex Overt Response	**A5**	Internalize
C6	Creating/Designing	**P6**	Adaption		
		P7	Origination		

Instructions for the students to conduct Sessional Class

SL	Instructions
1.	Experiment list for the sessional has been displayed. Each experiment is given a number.
2.	For each day, every student must have to implement the given experimental task according to experiment number. Marks will be given by the faculty for the implemented experiment.
3.	Students may be permitted to carry out missed experiments in their own time with the approval of faculty only.
4.	Each sessional report must be submitted within 15 minutes after starting the class time by wrapping a cover file. Report must be hand written format. Each report will contain the following parts- a. Experiment No b. Name of the Experiment c. Problem Description d. Algorithm / Flowchart e. Source Code f. Input g. Output h. Discussion (This part must be unique)
5.	Sessional marks awarded at the end of the semester will be strictly followed according to the sessional assessment method.

Experiment No: 01

Experiment Name: Classes and object

SL	Assigned Tasks	CO
Q1.	Write a C++ program using class and object.	1
Q2.	In C++, an object is created from a class. We have already created the class named MyClass, so now we can use this to create objects. To create an object of MyClass, specify the class name, followed by the object name	2

Experiment No: 02

Experiment Name: Access specifiers, static andnon-static members

SL	Assigned Tasks	CO
Q1.	Take two integer inputs from user. First calculate the sum of two then product of two. Finally, print the sum and product of both obtained results.	1,2
Q2.	Take value of length and breath of a rectangle from user as float. Find its area and print it on screen after type casting it to int.	1, 2

Experiment No: 03

Experiment Name: Constructors, Destructors and Copy Constructor

SL	Assigned Tasks	CO
Q1.	Dynamic initialization of object using constructor.	1
Q2.	Create a class named 'Rectangle' with two data members- length and breadth and a function to calculate the area which is 'length*breadth'. The class has three constructors which are : 1 - having no parameter - values of both length and breadth are assigned zero. 2 - having two numbers as parameters - the two numbers are assigned as length and breadth respectively. 3 - having one number as parameter - both length and breadth are assigned that number. Now, create objects of the 'Rectangle' class having none, one and two parameters and print their areas.	2

Experiment No: 04

Experiment Name: Default, parameterized, copy constructor, In-line functions

SL	Assigned Tasks	CO
Q1.	Develop a C++ program using constructor (Default constructor, parameterized constructor, copy constructor)	1
Q2.	Create a class 'Student' with three data members which are name, age and address. The constructor of the class assigns default values to name as "unknown", age as '0' and address as "not available". It has two functions with the same name 'setInfo'. First function has two parameters for name and age and assigns the same whereas the second function takes has three parameters which are assigned to name, age and address respectively. Print the name, age and address of 10 students.	2

Experiment No: 05

Experiment Name: Friend functions, static functions

SL	Assigned Tasks	CO
Q1.	Write a C++ program using Friend Function.	2
Q2.	Write a program which will ask the user to enter his/her marks (out of 100). Define a function that will display grades according to the marks entered as below:	3

Experiment No: 06

Experiment Name: Single inheritance

SL	Assigned Tasks	CO
Q1.	Write a C++ program using Inheritance.	2
Q2.	Create two classes named Mammals and MarineAnimals. Create another class named BlueWhale which inherits both the above classes. Now, create a function in each of these classes which prints "I am mammal", "I am a marine animal" and "I belong to both the categories: Mammals as well as Marine Animals" respectively. Now, create an object for each of the above class and try calling 1 - function of Mammals by the object of Mammal 2 - function of MarineAnimal by the object of MarineAnimal 3 - function of BlueWhale by the object of BlueWhale 4 - function of each of its parent by the object of BlueWhale.	3

Online Test-01

Experiment Name: Multiple inheritances, Multilevel Inheritance

SL	Assigned Tasks	Marks
Q1.	Create an Order class for a nightclub that contains a table number, a server's name, the number of patrons at the table, and a data member for the table minimum charge, which is $4.75. The class should include: [CO2, PO2, C3] i. A member function to display the table minimum charge. ii. A constructor to initialize an Order object. iii. A member function to display the order details, including the total minimum charge (minimum charge × number of patrons). Write a main () function that creates an Order object, assigns appropriate values, and displays the order details, including the total minimum charge. Sample output: Table Minimum Charge: $4.75 Table Number: 5 Server Name: John Doe Number of Patrons: 3 Total Minimum Charge: $14.25	10

Q2.	Create a RestaurantMeal class that holds the name and price of a food item served by a restaurant. Its constructor requires arguments for each field. Create a HotelService class that holds the name of the service, the service fee, and the room number to which the service was supplied. Its constructor also requires arguments for each field. Create a RoomServiceMeal class that inherits from both RestaurantMeal and HotelService. Whenever you create a RoomServiceMeal object, the constructor assigns the string “room service” to the name of the service field, and $4.00 is assigned to the service fee inherited from HotelService. Include a RoomServiceMeal function that displays all of the fields in a RoomServiceMeal by calling display functions from the two parent classes. Additionally, the display function should display the total of the meals plus the room service fee. In a main() function, instantiate a RoomServiceMeal object that inherits from both classes. For example, a “steak dinner” costing $19.99 is a “room service” provided to room 1202 for a $4.00 fee. [CO3, PO3, C3] Sample Output: Room Service Meal Details: Meal: Steak Dinner Price: $19.99 Service: room service Service Fee: $4.00 Room Number: 1202 Total Cost: $23.99	15

Experiment No: 07

Experiment Name: Multiple inheritances, Multilevel Inheritance

SL	Assigned Tasks	CO
Q1.	Write a C++ program using hierarchical and multilevel inheritance. C++ Multilevel Inheritance	2,3
Q2.	Create a class named Shape with a function that prints "This is a shape". Create another class named Polygon inheriting the Shape class with the same function that prints "Polygon is a shape". Create two other classes named Rectangle and Triangle having the same function which prints "Rectangle is a polygon" and "Triangle is a polygon" respectively. Again, make another class named Square having the same function which prints "Square is a rectangle". Now, try calling the function by the object of each of these classes.	2,3

Experiment No: 08

Experiment Name: Operator Overloading

SL	Assigned Tasks	CO
Q1.	Write a C++ program using operator overloading (Unary operator). Syntax for C++ Operator Overloading	2,3
Q2.	Develop a C++ program to Overload the coord class so it can use coord objects in operations in which an integer value can be multiplied by each coordinate. Allow the operations to use either order: ob * int or int * ob.	3,4

Experiment No: 09

Experiment Name: Operator Overloading

SL	Assigned Tasks	CO
Q1.	Write a C++ program using operator overloading (Binary operator). Syntax for C++ Operator Overloading	2,3
Q2.	Develop a C++ program to Overload the + for the three d class in Question 2 so that it accepts the following types of operations: ob + int ; int + ob;	3,4

Experiment No: 10

Experiment Name: Abstract classes, virtual functions and overriding

SL	Assigned Tasks	CO
Q1.	Write a C++ program using abstract class.	2,3
Q2.	Suppose you have a Piggie Bank with an initial amount of $50 and you have to add some more amount to it. Create a class 'AddAmount' with a data member named 'amount' with an initial value of $50. Now make two constructors of this class as follows: 1 - without any parameter - no amount will be added to the Piggie Bank 2 - having a parameter which is the amount that will be added to the Piggie Bank Create an object of the 'AddAmount' class and display the final amount in the Piggie Bank.	3,4

Experiment No: 11

Experiment Name: Exception Handling, Object Oriented I/O, Conversion Function

SL	Assigned Tasks	CO
Q1.	Write a C++ program using exception handling.	2,3
Q2.	Create a Job class that holds a Job ID number and the cost of the Job. Create a JobExceptionclass that holds a Job and an error message. When the user enters Job data, if the Job fee is below $250, then create a JobException object and throw it. Write a main()function that declares an array of eight Job objects. If a JobException object is thrown during the data entry for any Job, require the user to enter data for a new Job, and replace the invalid Job. Save the file as Job.cpp	3,4

Experiment No: 12

Experiment Name:File Handling, Template functions, Overview of Standard Template Library

SL	Assigned Tasks	CO
Q1.	Write a C++ program using template function.	2,3
Q2.	Initialize a vector of integers called num with the values {1, 2, 3, 4, 5, 8}. Then modifies the values at the 2nd and 5th positions of the num vector to be 9 and 7, respectively. It then prints out the modified vector. After that, the code erases the elements in the vector starting from the 3rd position until the end of the vector. It then prints out the remaining elements in the vector. Sorting in ascending order over the vector elements using a for loop. Demonstrates the code using of vectors in C++.	3,4

Sample Online Test-02

Total Marks: 25

SL No. | **Online Test-02** | Marks

1. eate an Investment class that contains fields to hold the initial value of an investment, the current value, the profit (calculated as the difference between current value and initial value), and the percent profit (the profit divided by the initial value). Include a constructor that requires initial and current values and a display function. Create a House class that includes fields for street address and square feet, a constructor that requires values for both fields, and a display function. Create a HouseThatIsAnInvestment class that inherits from Investment and House. It includes a constructor and a display function **5**

that calls the display functions of the parents. Write a main()function that declares a HouseThatIsAnInvestment and displays its values.

2. Complete the following tasks: **10**

a. Create a NonNumericException class that inherits from runtime_error. The class contains a string that holds any non-numeric data entered and uses its parent's what() function to display an appropriate error message.

b. Create a class that holds customer orders. Include an order number, quantity ordered, price each, and total which will be calculated as quantity times price so that the following occur:
» If any numeric data is not entered as a number, throw a NonNumericException.
» If the entered order number is numeric but more than four digits, throw an appropriate message.
» If the entered order quantity is over 50, throw the quantity.
» If the entered price each is more than $39.95, throw the price.
» If the total (quantity times price) is more than $1000.00, throw the Order object.

c. Develop a C++ program to create an array of five orders. Accept data for each one from the user. Catch and handle the errors as follows:
» If the order number was numeric but more than four digits, display a message and force the user to renter the number.
» If the order total was more than the $1000 limit, accept the Order but display a message asking the order taker to check the customer's credit limit.
» If any other exception occurs, display an appropriate message and set all the order values to zeroes.
» If no exceptions occur, accept the Order.
» After five Orders are accepted, display them.

3. Suppose we have a class called BankAccount with a balance attribute and methods for depositing and withdrawing money from the account. We want to create two subclasses of BankAccount: SavingsAccount and CheckingAccount. **10**

For SavingsAccount, we want to override the withdraw() method to ensure that the balance in the account does not go below a certain minimum amount. We also want to provide an overloaded version of the withdraw() method that takes an additional parameter for a withdrawal fee.

For CheckingAccount, we want to provide an overloaded version of the withdraw() method that takes an additional parameter for the maximum amount that can be withdrawn in a single transaction. We also want to provide an overdraft() method that allows the account to go into negative balance up to a certain limit.

Complex Engineering Problem

You are tasked with creating a simple banking system using object-oriented programming (OOP) principles in C++. The system should allow customers to create accounts, deposit and withdraw funds, and check their account balance. Additionally, bank employees should be able to view account information.
Here are the main features of the system:

1. Each customer should have the following attributes:
    - Customer ID (a unique integer)
    - Customer name (a string)
    - Account balance (a floating-point number)
2. You need to implement two classes:
    - **Customer**: This class should represent a bank customer and should have private member variables for customer ID, customer name, and account balance. It should also have the following public member functions:
        - A constructor to initialize the customer with a customer ID, customer name, and initial account balance.
        - Functions to deposit and withdraw funds from the customer's account.
    - **Bank**: This class should represent the bank and should have a data structure to store customer objects. It should have the following public member functions:
        - A function to add an amount customer to the bank.
        - A function to withdraw money from customer to the bank.(If Possible, Otherwise Show invalid transaction)
        - A friend class **BankEmployee** to allow bank employees to access customer account information. Bank employees should be able to view the customer's name, customer ID, and account balance.
3. You need to create a **BankEmployee** class with a friend function to display the account information of a customer. The **BankEmployee** class should not have any other functionality.

Step-1: Define the problem

- Problem Statement:
    - Create a simple banking system using C++ and OOP principles.
    - Implement two classes: Customer representing bank customers and Bank representing the bank itself.
    - Allow customers to create accounts, deposit and withdraw funds, and check their account balance.
    - Enable bank employees to view customer account information using a friend class.

Range of Conflicting's Requirements (P2)

- Customer Privacy vs. Employee Access
- User-Friendliness vs. Security
- Transaction Speed vs. Data Integrity
- Scalability vs. Performance
- Flexibility vs. Standardization
- Real-Time Updates vs. System Load
- Concurrent Access vs. Data Consistency
- Error Handling vs. User Experience:
- Transaction Confirmation vs. Latency
- Regulatory Compliance vs. Customer Convenience

Step-2: Gather Information

- Customer Attributes:
  - Customer ID (unique integer)
  - Customer name (string)
  - Account balance (floating-point number)
- Customer Class Functions:
  - Constructor to initialize customer with ID, name, and initial balance.
  - Functions to deposit and withdraw funds.
- Bank Class Functions:
  - Function to add a customer to the bank.
  - Function to withdraw money from a customer (if possible).
  - Friend class BankEmployee to view customer account information.
- BankEmployee Class:
  - Friend function to display customer account information.

Depth of Analysis Required (P3) for Customer Attributes:

- Validate Uniqueness of Customer IDs
- Handle Edge Cases for Customer Name (Empty String, Special Characters)
- Implement Constraints on Account Balance (e.g., Non-Negative)
- Consider Data Types and Storage Precision for Floating-Point Numbers

Depth of Analysis Required (P3) for Customer Class Functions:

- Ensure Initialization of Customer Attributes in the Constructor
- Implement Validation for Deposit and Withdrawal Amounts
- Consider Handling Overdrafts in Withdrawal Function

Depth of Analysis Required (P3) for Bank Class Functions:

- Implement Logic to Prevent Duplicate Customer IDs
- Consider Error Handling for Attempted Withdrawals from Insufficient Balances
- Validate Customer Existence Before Performing Operations

Depth of Analysis Required (P3) for BankEmployee Class:

- Restrict Access to Sensitive Customer Information
- Ensure Read-Only Access for Displaying Customer Account Information

Step-3: Generate Multiple Solutions

- Customer Class Solution:
  - Implement a class with private member variables for ID, name, and balance.
  - Provide constructor and functions for deposit and withdrawal.
- Bank Class Solution:
  - Implement a class with a data structure to store customer objects.
  - Add functions to add customers and withdraw money from customers.
- BankEmployee Class Solution:
  - Create a friend class with a function to display customer account information.

Depth of Analysis (P3) for Customer Class Solution:

- Validate Constructor Input Parameters
- Ensure Encapsulation of Private Member Variables

- Consider Handling Floating-Point Precision in Balance Operations

Familiarity of Issues (P4) for Customer Class Solution:

- Understanding of Constructor Initialization Order
- Awareness of Data Encapsulation and Access Control
- Familiarity with Floating-Point Arithmetic Challenges

Depth of Analysis (P3) for Bank Class Solution:

- Design Considerations for Data Structure Efficiency
- Validate Customer Existence Before Withdrawal
- Address Concurrency Issues in Customer Data Storage

Familiarity of Issues (P4) for Bank Class Solution:

- Understanding of Data Structure Selection (e.g., Array, Linked List)
- Consideration of Transaction Handling and Rollback
- Familiarity with Thread-Safety and Concurrent Access Challenges

Depth of Analysis (P3) for BankEmployee Class Solution:

- Ensure Limited Access to Sensitive Customer Information
- Implement Secure Communication Channels for Employee Access
- Validate Employee Authorization Before Displaying Information

Familiarity of Issues (P4) for BankEmployee Class Solution:

- Understanding of Friend Classes and Their Limitations
- Awareness of Security Concerns in Employee Access
- Familiarity with Authorization and Authentication Mechanisms

Step-4: Analyze and Select a Solution

- Customer Class Analysis:
  - Pros: Encapsulates customer data and operations.
  - Cons: May lack validation for negative balances.
- Bank Class Analysis:
  - Pros: Manages a collection of customers.
  - Cons: Withdrawal logic needs careful handling to avoid negative balances.
- BankEmployee Class Analysis:
  - Pros: Enables restricted access to customer information.
  - Cons: Limited functionality, focusing solely on account information display.

Depth of Knowledge Required (P1) for Customer Class Analysis:

- Understanding of Object-Oriented Principles
- Familiarity with C++ Language Features for Class Implementation
- Knowledge of Constructors and Member Functions in C++

Extent of Applicable Code (P5) for Customer Class Analysis:

- Code for Encapsulation of Customer Data
- Code for Constructor Initialization and Member Function Implementation
- Code for Handling Validations in Deposit and Withdrawal Functions

Depth of Knowledge Required (P1) for Bank Class Analysis:

- Understanding of Data Structures for Customer Management
- Knowledge of Transaction Handling and Atomic Operations
- Awareness of Concurrency Issues in Data Storage

Extent of Applicable Code (P5) for Bank Class Analysis:

- Code for Customer Addition to Data Structure
- Code for Validations and Handling in Withdrawal Function
- Implementation of Concurrency Handling Mechanisms

Extent of Applicable Code (P5) for BankEmployee Class Analysis:

- Code for Friend Class Implementation
- Code for Secure Information Display Function
- Implementation of Authorization Checks in Friend Function

Extent of Stakeholder Involvement and Level of Conflicting Requirements (P6) for BankEmployee Class Analysis:

- Involvement of Bank Employees in System Access
- Potential Conflict between Employee Access and Customer Privacy
- Stakeholder Input for Defining Access Levels and Displayed Information

Step-5: Test and Implement a Solution

- Testing:
  - Test customer creation, deposit, and withdrawal functions.
  - Validate bank functions for adding customers and handling withdrawals.
- Implementation:
  - Code the customer, bank, and bank employee classes in C++.
  - Ensure proper encapsulation, error handling, and adherence to OOP principles.
  - Test the overall system to verify functionality and data integrity.

Extent of Applicable Code (P5) for Testing:

- Develop Test Cases for Customer Creation, Deposit, and Withdrawal Functions
- Implement Test Cases for Bank Functions, Including Customer Addition and Withdrawal Handling

Interdependence (P7) for Testing and Implementation:

- Iterative Process: Testing informs code improvements, and the code influences test case development.
- Ongoing Validation: Testing validates the implementation, and implementation correctness impacts testing outcomes.

Approved by

PSAC Committee